

6. Assignment

Complex Systems for Bioinformaticians

Deadline: May 26, 12:00 (**before** the lecture)

The homework should be worked out individually or in groups of 2 students. Pen & paper exercises should be handed in at the designated deadline. Each solution sheet must contain the names and ‘Matrikelnummern’ of all group members. Please staple all sheets.

Programming exercises must be submitted via Whiteboard.

Homework 1 (Pen/paper + Implementation (upload via Whiteboard), 1+2 points)

You are given the following ODE-system:

$$\begin{aligned}\frac{d}{dt}x_1 &= -k_1 \cdot x_1 \cdot x_2 \\ \frac{d}{dt}x_2 &= -k_1 \cdot x_1 \cdot x_2 + k_2 + k_3 \cdot x_2 \cdot x_3 \\ \frac{d}{dt}x_3 &= -k_3 \cdot x_2 \cdot x_3\end{aligned}$$

- a) (**pen & paper**) Derive the stoichiometric matrix and rate functions $r_1, \dots$.
- b) (**to be uploaded via Whiteboard**) Write a small program implementing this model.
- (i) Your program must write the stoichiometric matrix into a text file named “SMatrix.txt”. Columns should be comma separated, e.g. using the numpy function ‘savetxt’ with a comma delimiters (‘,’). Write numbers as signed integers, e.g. ‘-1’.
- (ii) Compute the system state after one time step. Use the parameters $k_1 = 0.01$, $k_2 = 0.1$, $k_3 = 0.01$, time step $\Delta t = 1$, and the current system state, which is given in the “Input.txt” file:

$$\begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 5 \\ 25 \\ 15 \end{pmatrix}$$

Your program should write the output into a text file named “ODEValue.txt”. Round the output to the first two digits after the comma, e.g. ‘1.20’ or ‘0.34’.

Name your program “Ex2_1.py” and submit via Whiteboard.

Hint: There is a code scaffold (CodeScaffold1.py) provided for you in Whiteboard which you can adapt.

Homework 2 (Implementation (upload via Whiteboard), 4 points)

Write a program implementing the model above and generate trajectories using the stochastic simulation algorithm (SSA, a.k.a. Gillespie’s algorithm). The program should read the input file (“Input2.txt”) provided in Whiteboard. The first number in the input file is the ‘seed’ of the random number generator, and the second is the number of trajectories N to be computed. Using this input, compute the trajectories for N simulations up to time $T = 1$. For each simulation, write the time and the values for X_2 into a file “Task2TrajY.txt”, where ‘Y’ = $1 \dots N$ is the current simulation (e.g. “Task2Traj3.txt” contains the jump-times and the trajectory of X_2 from the third simulation). The output textfile should be in comma-separated text format using 3 digits after the comma (format ‘%1.3f’), e.g.

0.000,0.010,0.017,0.202, ..., 9.763
5.000,6.000,5.000,6.000, ..., 12.000

where the first row are the times $t^{(i)} \leftarrow t^{(i-1)} + \Delta t^{(i)}$ and the second row contains the corresponding number of X_2 at that time $X_2(t^{(i)})$. Name your program “Ex2_2.py” and submit via Whiteboard.

Hint: There is a code scaffold (CodeScaffold2.py) provided for you in Whiteboard which you can adapt.

Homework 3 (Implementation (upload via Whiteboard), 4 points)

Implement the EXTRANDE method for the model from Homework 2 with the following modification:

$$r_2(t) = k_2 \cdot 0.5 \cdot (\sin(t \cdot 180) + 2)$$

where ‘sin’ denotes the sine function.

Again, the program should read the input file (“Input2.txt”) provided in Whiteboard, containing the seed for the random-number generator and the number of trajectories N . Compute the trajectories for N simulations up to time $T = 10$. For each simulation, write the time and the values of X_2 into a file “Task3TrajY.txt”, for $Y = 1 \dots N$. The output textfile should be in comma-separated text format using 3 digits after the comma (format ‘%1.3f’). Name your program “Ex2_3.py” and submit via Whiteboard.

Hint: First explore the values of $k_2 \cdot 0.5 \cdot (\sin(t \cdot 180) + 2)$ over the time horizon $t \in [0, \infty]$ and get their maximum to obtain the upper bound B .

Good luck!